

Project Plan

Fake News Detection Using Advanced NLP – Internship Project

1. Project Overview

This project plan defines the purpose, scope, phased structure, resourcing, and risk approach for building a fake news detection system, from raw dataset acquisition through a deployed Streamlit demo. It complements the Action Plan (which focuses on the week-by-week execution timeline) by documenting the project's scope boundaries, phase-gate review structure, and roles more formally, in the style of a project charter.

1.1 Purpose

To design, build, evaluate, and deploy a content-based fake news classifier using classical machine learning and NLP techniques, producing both a working system and a complete set of internship-quality supporting documents (research report, action plan, final report, and insights document).

1.2 Objectives

- Deliver a working classifier comparing Naive Bayes, SVM, and Random Forest across TF-IDF and Word2Vec feature representations.
- Produce reproducible, well-documented notebooks covering the full pipeline from raw data to saved model.
- Ship a usable Streamlit demo of the best-performing model.
- Maintain an industry-standard repository structure suitable for a public portfolio.

2. Scope

In scope	Out of scope
EDA, preprocessing, TF-IDF & Word2Vec feature engineering	Deep learning / transformer-based models (BERT, LSTM)
Naive Bayes, SVM, Random Forest training and comparison	Multilingual or non-English datasets
Model evaluation (accuracy, precision, recall, F1, ROC-AUC)	Real-time social-media ingestion or live data pipelines
Streamlit web application for single-article predictions	User authentication, hosting, or production-scale deployment

Research report, action plan, final report, insights document	Formal ethics board review or external audit
---	--

3. Work Breakdown Structure

The project is broken into 14 phases, each independently reviewable before the next begins:

#	Phase	Output
1	Project understanding	Problem statement, objectives, workflow diagram
2	Dataset selection	ISOT dataset identified and justified
3	Research report	15+ page theoretical report
4	Action plan	Timeline, risks, resources
5	Exploratory data analysis	EDA notebook and visualizations
6	Text preprocessing	Cleaning/tokenization/lemmatization notebook
7	TF-IDF feature engineering	TF-IDF notebook and vectorizer
8	Word2Vec feature engineering	Word2Vec notebook and model
9	Train/test split	Stratified 80/20 split, saved arrays
10	Model training	Six trained models (3 algorithms × 2 feature sets)
11	Model evaluation	Metrics, confusion matrices, ROC curves, recommendation
12	Save best model	Packaged model + vectorizer + metadata
13	Streamlit app	Deployed prediction interface
14	Documentation	requirements.txt, README, GitHub polish

4. Roles and Responsibilities

Role	Responsibility
Intern (project owner)	Executes each phase, reviews outputs, approves progression to the next phase
Mentor / reviewer (Claude, this session)	Drafts each phase's deliverable, explains theory and code, flags risks and best practices
Academic supervisor (if applicable)	Reviews final submission against the internship's grading rubric

5. Phase-Gate Review Process

Each phase is completed and presented in full before the next begins, rather than batching multiple phases together. This mirrors a real-world sprint-review process: work is checked at each gate, mistakes are caught early rather than compounding across phases, and the project owner retains a natural checkpoint to redirect scope if priorities shift.

6. Resources and Tools

Category	Resource
Language / runtime	Python 3.10+, Jupyter Notebook
Core libraries	pandas, numpy, nltk, scikit-learn, gensim
Visualization	matplotlib, seaborn, wordcloud
Deployment	Streamlit
Version control	Git and GitHub
Compute	Local machine or Google Colab
Dataset source	Kaggle – ISOT Fake and Real News Dataset

7. Risk Management

Risk	Owner	Mitigation
Dataset artifacts inflate accuracy	Intern	Strip source-identifying patterns in preprocessing; disclose in reports
Word2Vec underperforms small in-domain corpus	Intern	Compare directly against TF-IDF; consider pretrained vectors if needed
Timeline slips past internship deadline	Intern	Treat Streamlit app and full-length report as stretch goals if time is short
Model files/versions mismatch at deployment	Intern	Follow the Phase 12 deployment-readiness checklist before building the app

8. Success Criteria

- All 14 phases produce a reviewed, working deliverable.
- At least one model achieves competitive accuracy/F1 consistent with published benchmarks on this dataset (commonly 90%+ in prior literature).
- The Streamlit app runs end-to-end on a fresh environment using only the files in the

repository.

- The repository is structured so a third party could clone it, follow the README, and reproduce the results.

9. Assumptions and Constraints

9.1 Assumptions

- The intern has continuous access to a laptop or Colab environment capable of running Python 3.10+ and installing the required libraries.
- The ISOT dataset, once downloaded, does not require additional licensing or approval for internship/educational use.
- Roughly 10–15 hours per week of part-time effort are available; the six-week timeline in the Action Plan scales proportionally if actual availability differs.
- No GPU is required – all three algorithms and Word2Vec training run acceptably on CPU at this dataset size.

9.2 Constraints

- The project must use only the ISOT dataset unless a documented reason justifies switching sources mid-project.
- Modeling is scoped to classical ML (Naive Bayes, SVM, Random Forest); transformer-based models are explicitly out of scope for this iteration.
- The Streamlit app is a local/demo-grade deployment, not a production service with authentication, monitoring, or uptime guarantees.
- Internship deadlines take priority over scope – if time runs short, the Deliverables document's required items (see Section 2) take precedence over stretch goals like the 15–20 page report length.

10. Quality Assurance Approach

Quality is maintained through a small number of concrete practices applied consistently across every phase, rather than a single end-of-project review:

- Every notebook cell that trains or transforms data is preceded by a markdown explanation of why the step is being taken, so correctness can be checked against stated intent, not just against whether the code runs.
- Every notebook includes an explicit 'Common Mistakes' section calling out the most

likely way that specific phase goes wrong (e.g. data leakage, mismatched feature types), turning known failure modes into a checklist rather than relying on memory.

- Model comparisons always use the same train/test split (Section 5.4) and the same random seeds, so results are attributable to genuine modeling differences rather than split variance.
- Each phase's output is reviewed and approved before the next phase begins (see Section 5, phase-gate review), preventing errors from silently compounding across three or four downstream notebooks before being caught.
- The deployment-readiness checklist in Phase 12 is treated as a hard gate before the Streamlit app is built, not an optional afterthought.

11. Communication Plan

Checkpoint	Purpose	Format
End of each phase	Review deliverable, confirm before proceeding	In-session review (chat/notebook walkthrough)
End of Week 2 (post-EDA)	Confirm dataset quality and class balance before committing further effort	Short written summary
End of Week 5 (post-evaluation)	Confirm model choice before building the deployment app	Metrics table + recommendation
Project close	Final report and insights document handoff	Written deliverables (this document set)

12. Monitoring and Control

Progress is tracked against the Milestones and Timeline table in Section 3 (and the more granular weekly breakdown in the Action Plan). If a phase overruns its planned week, the response is to first check whether the overrun is due to a genuine blocker (e.g. a broken dataset download, an environment issue) versus scope creep (e.g. expanding a phase beyond what's needed for the next one to start). Genuine blockers are escalated immediately rather than absorbed silently; scope creep is deferred to the Future Scope section of the Final Report rather than allowed to delay the current phase.

13. Detailed Milestone Descriptions

Expanding on the Work Breakdown Structure in Section 3, each phase has a specific definition of done:

- Phases 1–2 (understanding, dataset): done when the problem statement, objectives, and dataset justification are written and the raw CSV files are present in dataset/.
- Phases 3–4 (research report, action plan): done when both documents are complete, reviewed, and exported to PDF.
- Phase 5 (EDA): done when class balance, missing/duplicate checks, length distributions, and word-frequency analysis are all computed and visualized, with plots saved to images/.
- Phase 6 (preprocessing): done when a single `clean_text_pipeline` function runs end-to-end on the full dataset and `cleaned_news.csv` is saved.
- Phases 7–8 (feature engineering): done when both the TF-IDF vectorizer and the Word2Vec model are trained and their outputs sanity-checked (mini-example for TF-IDF, nearest-neighbor check for Word2Vec).
- Phase 9 (split): done when train/test arrays for both feature sets share the same underlying row indices and the fitted TF-IDF vectorizer is saved.
- Phase 10 (training): done when all six model/feature combinations are trained without error and saved to disk.
- Phase 11 (evaluation): done when the metrics table, confusion matrices, ROC curves, and a data-driven best-model recommendation are all produced.
- Phase 12 (save best model): done when the deployment-readiness checklist is fully checked off.
- Phase 13 (webapp): done when the Streamlit app runs locally, accepts text input, and returns a label and confidence score without errors.
- Phase 14 (documentation): done when `requirements.txt`, `README.md`, and `.gitignore` are complete and the repository matches the structure defined in Section 1.